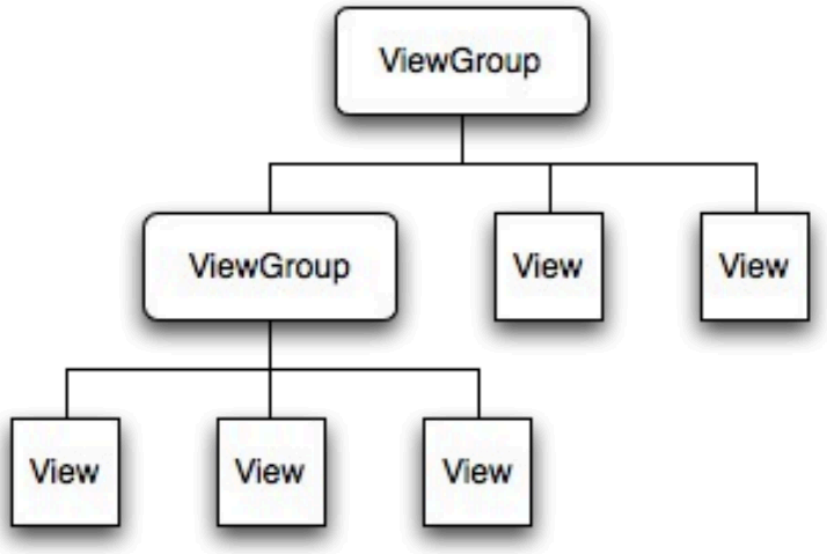
UI di un’activity

Description	UI per presentare informazioni all’utente / raccogliere Input

Una UI Android è un albero con foglie di classe View e nodi intermedi di classe **ViewGroup (**Come già visto, tipicamente definito in XML)

Ogni View è una classe Java con nome uguale al tag XML relativo e la disposizione visuale
delle view è regolata da un Layout



Una `ViewGroup` è una classe base che rappresenta un contenitore di visualizzazioni (Views). Essenzialmente, una `ViewGroup` è un tipo di `View` che può contenere altre `View` e `ViewGroup` all'interno di se stessa.

Le `ViewGroup` sono spesso utilizzate per organizzare e posizionare le visualizzazioni all'interno dell'interfaccia utente di un'applicazione Android.

Le `ViewGroup` sono sottoclassi della classe `View`, e le visualizzazioni contenute all'interno di una `ViewGroup` possono essere di qualsiasi tipo, inclusi widget come bottoni, testi, immagini, ma anche altre `ViewGroup`. Le `ViewGroup` forniscono metodi per aggiungere, rimuovere e organizzare le visualizzazioni figlie.

Alcune delle sottoclassi comuni di `ViewGroup` in Android:

1. `LinearLayout`:
    - Organizza le visualizzazioni in una singola riga o colonna, con un'orientazione specificata (orizzontale o verticale).
2. `RelativeLayout`:
    - Posiziona le visualizzazioni in relazione alle loro visualizzazioni genitore o tra di loro. È utile per la creazione di layout più complessi.
3. `FrameLayout`:
    - Contiene una singola visualizzazione, posizionata nello stack in modo che possa sovrapporsi ad altre visualizzazioni.
4. `ConstraintLayout`:
    - Un layout flessibile che consente di creare relazioni tra le visualizzazioni in modo più complesso rispetto a `RelativeLayout`.
5. `GridLayout`:
    - Organizza le visualizzazioni in una griglia bidimensionale, utile quando si desidera disporre le visualizzazioni in righe e colonne.
6. `ScrollView`:
    - Un tipo di `FrameLayout` che consente agli utenti di scorrere il suo contenuto se questo supera le dimensioni dello schermo.

In Android, i `LayoutManager` sono utilizzati per organizzare la disposizione e la visualizzazione degli elementi all'interno di un `RecyclerView`.

Un `RecyclerView` è un componente di interfaccia utente che visualizza un elenco o una griglia di elementi, ed è ampiamente utilizzato per la visualizzazione di elenchi di dati dinamici.

Ecco alcuni dei `LayoutManager` comuni utilizzati in Android:

1. LinearLayoutManager:

- **Descrizione:** `LinearLayoutManager` organizza gli elementi in un elenco lineare, orizzontale o verticale.
- **Utilizzo tipico:** Adatto per elenchi ordinati in una singola direzione.

```
RecyclerView recyclerView = findViewById(R.id.recyclerView);
LinearLayoutManager linearLayoutManager = new LinearLayoutMan
ager(this);
recyclerView.setLayoutManager(linearLayoutManager);
```

2. GridLayoutManager:

- **Descrizione:** `GridLayoutManager` organizza gli elementi in una griglia, consentendo specifiche per il numero di colonne.
- **Utilizzo tipico:** Adatto per la visualizzazione di elementi in una griglia.

```
RecyclerView recyclerView = findViewById(R.id.recyclerView);
GridLayoutManager gridLayoutManager = new GridLayoutManager(t
his, 2); // Numero di colonne
recyclerView.setLayoutManager(gridLayoutManager);
```

3. StaggeredGridLayoutManager:

- **Descrizione:** `StaggeredGridLayoutManager` organizza gli elementi in una griglia simile a `GridLayoutManager`, ma le righe possono avere altezze diverse.
- **Utilizzo tipico:** Adatto quando si desidera una griglia con elementi di altezza variabile.

```
RecyclerView recyclerView = findViewById(R.id.recyclerView);
StaggeredGridLayoutManager staggeredGridLayoutManager = new S
taggeredGridLayoutManager(2, StaggeredGridLayoutManager.VERTI
CAL);
recyclerView.setLayoutManager(staggeredGridLayoutManager);
```

4. LinearLayoutManager orizzontale:

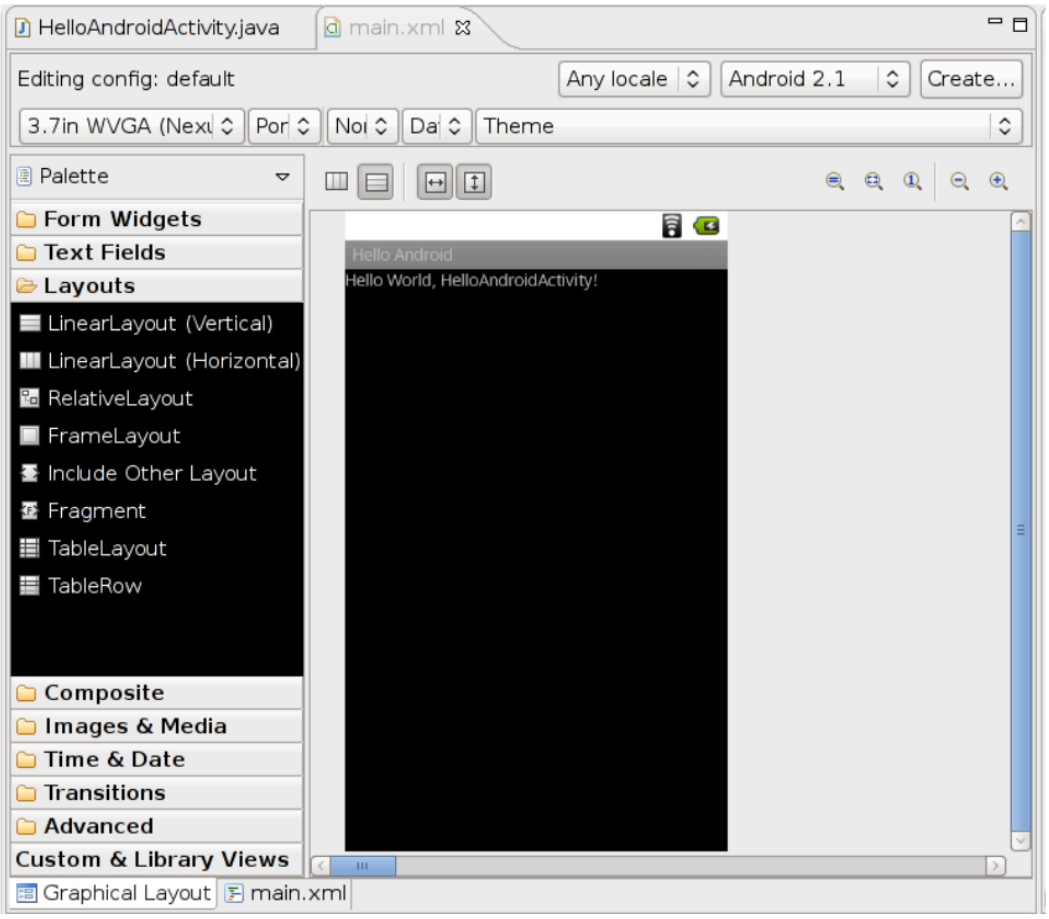
- **Descrizione:** `LinearLayoutManager` può essere configurato per disporre gli elementi orizzontalmente.
- **Utilizzo tipico:** Adatto per elenchi orizzontali.

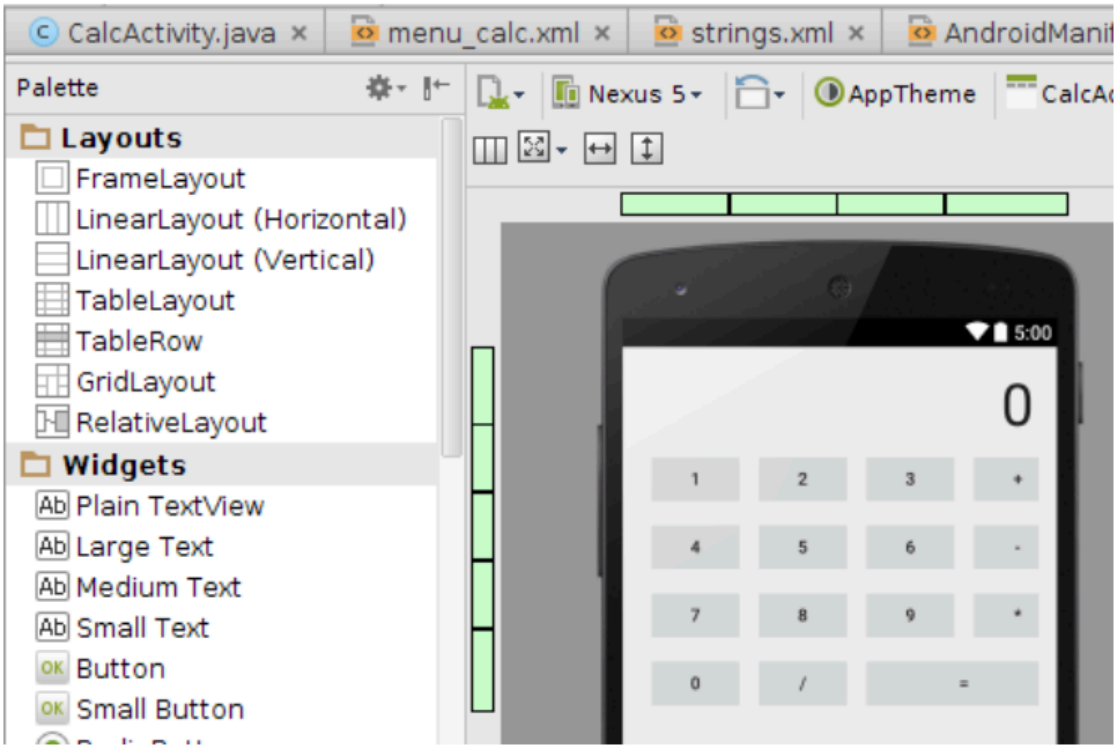
```
RecyclerView recyclerView = findViewById(R.id.recyclerView);
LinearLayoutManager linearLayoutManager = new LinearLayoutMan
ager(this, LinearLayoutManager.HORIZONTAL, false);
recyclerView.setLayoutManager(linearLayoutManager);
```

5. Custom LayoutManager:

Se nessuno dei `LayoutManager` incorporati soddisfa le tue esigenze, è possibile implementare un `LayoutManager` personalizzato estendendo la classe `RecyclerView.LayoutManager`. Questo offre pieno controllo sulla disposizione degli elementi all'interno di un `RecyclerView`.

```
public class CustomLayoutManager extends RecyclerView.LayoutM
anager {
    // Implementa il tuo layout qui
}
```


HelloAndroidActivity.java
main.xml
Editing config: default
Any locale
Android 2.1
Create...
3.7in WVGA (Nexu
Por
Noi
Da
Theme
Palette
Form Widgets
Text Fields
Layouts
LinearLayout (Vertical)
LinearLayout (Horizontal)
RelativeLayout
FrameLayout
Include Other Layout
Fragment
TableLayout
TableRow
Composite
Images & Media
Time & Date
Transitions
Advanced
Custom & Library Views
Hello Android
Hello World, HelloAndroidActivity!
Graphical Layout
main.xml



Listener

Un "listener" è un oggetto o una funzione che viene utilizzato per ascoltare e rispondere a eventi o notifiche che si verificano in un'applicazione o in un sistema.

Il concetto di "listener" è ampiamente utilizzato in diversi contesti, tra cui sviluppo di interfacce utente, gestione di eventi e programmazione asincrona.

Ecco alcune delle situazioni più comuni in cui vengono utilizzati i listener:

1. Interfacce Utente (UI):

- Nei framework per lo sviluppo delle interfacce utente, come Android o Swing (Java), i listener vengono utilizzati per gestire eventi come clic sui pulsanti, modifiche di testo, selezioni di elementi, ecc.

```
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
```

```
        // Codice da eseguire quando il pulsante viene clicca
to
    }
});
```

2. Gestione di Eventi:

- In molte situazioni, i listener sono utilizzati per la gestione di eventi generati da dispositivi o sistemi esterni. Ad esempio, un listener può essere utilizzato per gestire gli eventi di clic del mouse, la pressione di tasti su una tastiera o i cambiamenti di orientamento su un dispositivo mobile.

```
button.addEventListener("click", new ClickListener() {
    @Override
    public void onClick(Event event) {
        // Codice da eseguire quando il pulsante viene clicca
to
    }
});
```

3. Programmazione Asincrona:

- Nei contesti di programmazione asincrona, i listener vengono spesso utilizzati per gestire la completamento di operazioni o la ricezione di dati. Ad esempio, nei callback di JavaScript o nelle risposte asincrone delle richieste HTTP.

```
fetchData(function(data) {
    // Codice da eseguire quando i dati sono stati recuperati
});
```

4. Observer Pattern:

- Il concetto di listener è spesso implementato nell'Observer Pattern, in cui un oggetto (l'osservato) mantiene una lista di "ascoltatori" (listener) che vengono notificati quando si verificano determinati eventi.

```
// Observer Pattern
subject.addObserver(new MyObserver() {
    @Override
    public void update() {
        // Codice da eseguire quando viene aggiornato lo stat
o dell'osservato
    }
});
```